Reviewer Pack — Hypergeometric Function Zeros and Boolean Circuit Depth

Entry 78e396735db4 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-29 09:20:49 UTC

Hypergeometric Function Zeros and Boolean Circuit Depth

Entry ID: `78e396735db4` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-29 09:20:49 UTC

1. Conjecture

Field A (mathematical branch): Hypergeometric Functions **Field B** (complexity object): Boolean Circuit Complexity

Statement:

> {'one': 'For any given Boolean circuit C with depth D, the number of zeros of the hypergeometric function F(z) = (1 - z)^(-D/2) * [∏_{i=1}^n [(1 + x_i/z)^(-1)]', 'two': 'is upper-bounded by a polynomial in n, the number of variables in C.', 'three': 'Equivalently, for any fixed D, there exists a constant c_D such that any Boolean circuit with depth D has at most c_D * 2^n gates.'}

Rationale (proposer's reasoning):

> {'one': 'Hypergeometric functions have been studied extensively in mathematics, but their direct application to complexity theory is rare.', 'two': "The zeros of a hypergeometric function can potentially reveal the structure of a Boolean circuit's computation.", 'three': 'This conjecture could provide a new way to analyze Boolean circuits, possibly leading to improved lower bounds on circuit depth.'}

Taxonomy category: `Hypergeometric Functions × Boolean Circuit Complexity` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `2bcad2c53b62482d`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> For Boolean circuits with depth D, if the number of zeros of F(z) exceeds a polynomial upper bound c_D * 2^n for any n or any seed yields a metric > 10, then the conjecture is falsified.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "hypergeometric function zeros" AND "Boolean circuit complexity" - "(1 - z)^(-D/2)" AND boolean circuit depth - "polynomial in n" AND hypergeometric functions Boolean circuits

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def hypergeometric_zeros(D, n):
    zeros = []
    for z in [Fraction(1 + i, 2) for i in range(1, 2*n+1)]:
        if abs(z) > 1e-6 and abs((1 - z)**(-D/2)) < 1e-6:
            zeros.append(z)
    return zeros

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n = random.randint(5, 40)
    D = random.randint(1, 10)
    c_D = 10 * D  # Example polynomial upper bound

    circuit = [random.choice([0, 1]) for _ in range(n)]
    zeros = hypergeometric_zeros(D, n)

    metric_value = len(zeros) / (c_D * 2**n)
    conjecture_holds = metric_value <= 1
    counterexample = "" if conjecture_holds else f"Too many zeros: {len(zeros)} > {c_D * 2**n}"

    return {
        "metric_name": "Zeros of Hypergeometric Function",
        "metric_value": metric_value,
        "instances_tested": n,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }
```

```python
if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    else:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"Too many zeros\" first_failing_seed={first_fail
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_2017a71d.py", line 53, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_2017a71d.py", line 33, in run_trial
    zeros = hypergeometric_zeros(D, n)
            ^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_2017a71d.py", line 21, in hypergeometri
    if abs(z) > 1e-6 and abs((1 - z)**(-D/2)) < 1e-6:
                              ~~~~~~~^^~~~~~
  File "/usr/lib/python3.12/fractions.py", line 829, in __pow__
    return float(a) ** b
           ~~~~~~~~~^^~~
ZeroDivisionError: 0.0 cannot be raised to a negative power
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test code crashed before producing data, which means we cannot confirm whether the number of zeros of F(z) exceeds a polynomial upper bound or not. | next: Re-run the test with proper error handling to ensure it completes and produces results.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11082
2	preregistration	ollama_remote	glm4:latest	0	0	5827
3	novelty	ollama_remote	glm4:latest	0	0	4681
4	novelty	ollama_remote	glm4:latest	0	0	5421
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14082
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9501
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7771
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	6975
9	judge	ollama_remote	glm4:latest	0	0	11886

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 77226 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/78e396735db4.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/78e396735db4.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/78e396735db4.tar.gz` (if generated)